

CO3 - AT1 - Analytical Problem Solving

i) Constructor and Destructor Execution order

Constructor 10

Constructor 20

Destructor 20

Destructor 10

ii) Order of constructor & destructor execution

\* Demo d1(10) is created first, so its constructor executes first.

\* Demo d2(20) is created second, In FIFO order

\* when main() ends, local objects are destroyed.

\* d2 was created last, so its destructor executes first

iii) Why are Destructors called in Reverse order?

Destructors are called in reverse order of object creation (LIFO). This ensures that the most recently created objects is destroyed first. It is useful because later-created objects may depend on resources or objects created earlier.

## 2) Constructor Overloading Resolution

- i) Analyze which constructor is called for different object declarations.

| Object Declaration               | Constructor called    | Reason               |
|----------------------------------|-----------------------|----------------------|
| Product P <sub>1</sub> ;         | Product()             | No arguments         |
| Product P <sub>2</sub> (10);     | Product(int x)        | One integer argument |
| Product P <sub>3</sub> (10, 20); | Product(int x, int y) | Two integer argument |

- ii) Predict the output

Default   One Param   Two Param

- iii) How constructor overloading is Resolved?

The compiler determines which constructor to call by checking no. of arguments, types of arguments, best matching parameter list.

Thus, the compiler selects the appropriate constructor at compile time based on the arguments supplied.



### 3) Operator Overloading for Complex Numbers

i) How the + operator works for objects.

The + operator is overloaded using the operator+(C) function. When two complex objects are added:

$$C_3 = C_1 + C_2$$

The compiler internally calls:

$$C_3 = C_1 \cdot \text{operator} + (C_2)$$

ii) Results of Adding Two Complex Objects.

Complex  $C_1(3, 4)$ ;

Complex  $C_2(2, 5)$ ;

Complex  $C_3 = C_1 + C_2$ ;

$$C_3 = 5 + 9i$$

iii) Operator overloading vs Function-Based Addition

| operator overloading               | Function-Based Addition          |
|------------------------------------|----------------------------------|
| * uses + directly between objects. | uses a normal function.          |
| * Example - $C_1 + C_2$            | Example - $\text{add}(C_1, C_2)$ |
| * Easy and natural to read         | More explicit.                   |

## 4) Shallow copy

### i) Problems with shallow copy.

- \* Shallow copy copies only the pointer addresses.
- \* Both objects share the same medical history.
- \* Changes in one object affect the other.
- \* It may cause double deletion and memory errors.

### ii) Data Corruption

It is copied patient record modifies the shared medical history, the original record is also changed, causing incorrect medical data.

### iii) Deep copy

Deep copy creates separate memory for each patient's medical history.

Original → History 1

Copy → History 2

Thus, changes in one record do not affect the other, ensuring safe and independent patient records.



## 5) Polynomial Class with operator overloading

### i) Advantage of operator overloading

- \* Polynomial addition can be written simply as  $P_3 = P_1 + P_2$ .
- \* It is easier to read than a function like  $\text{add}(P_1, P_2)$ .
- \* It makes the program more natural and simple.

### ii) Adding polynomials of Different Degrees

The coefficient at the same power are added.

Example:

$$P_1 = 3 + 2x + 5x^2$$

$$P_2 = 4 + 3x + 2x^2 + 6x^3$$

$$\text{Result} = 7 + 5x + 7x^2 + 6x^3$$

The polynomial with higher degree determines the size of the result.

### iii) Memory Handling

- \* Dynamic arrays must be allocated using `new`.
- \* Memory must be released using `delete[]`.
- \* Improper memory handling can cause memory leaks.

## 6) Static Member Behaviour

### i) Exact output

3454

### ii) Why Does the Value Remain?

Total Accounts is a static data member, so only one copy is shared by all account objects.

$$a_1, a_2, a_3 = 3$$

$$a_4 = 4$$

When  $a_4$  goes out of scope, the static variable is not destroyed or decremented because no destructor is defined. Therefore, the final value remains 4.

### iii) Why can `getTotal()` be called without an object?

`getTotal()` is a static member function, so it can be called using the class name:

```
Account::getTotal();
```

No object is required.

Static members are useful when data must be shared by all objects such as:

Total no. of bank accounts

Total employees

Global counters.



## 7) Friend Function vs Member Function

### i) How Friend Function Accesses Private Members.

A friend function is declared using the friend keyword inside the class. It can directly access the class's private and protected members.

$d_1$ .meters

$d_2$ .meters

and returns their sum.

### ii) Friend Function vs Member Function

| Friend Function                                      | Member Function                         |
|------------------------------------------------------|-----------------------------------------|
| Not a member of the class                            | Belongs to the class                    |
| Called as $\text{add}(d_1, d_2)$                     | Called as $d_1.\text{add}(d_2)$         |
| Can access private members                           | Can access private members directly.    |
| Useful for operations involving two or more objects. | Usually operates on the current object. |

### iii) When is Friend Function Appropriate?

A friend function is useful when an operation needs to access private data of multiple objects, such as adding two Distance objects.

## 8) Inheritance and Constructor Chaining

i) Exact output

ABC → C → B → A

ii) Order of Execution

Constructors: Base class → Derived class

A() → B() → C()

Destructors: Derived class → Base class

~C() → ~B() → ~A()

iii) Why This order?

- \* The base class constructor must execute first to initialize the inherited / base part of the object.
- \* Then the derived class constructors execute.
- \* During destruction, the derived class is destroyed first, followed by its base classes.
- \* This follows the LIFO principle.



## 9) Virtual Functions and Runtime Polymorphism.

i) Why area() must be virtual.

- \* area() is declared virtual in the base class.
- \* It allows the correct derived-class function to be called through a shape\* pointer.
- \* Thus, Circle, Rectangle, Triangle can calculate their own areas.

ii) If area() is not virtual

The base class area() would be called instead of the derived-class version when using a shape\* pointer.  
So runtime polymorphism would not work correctly.

iii) Runtime Polymorphism and performance

- \* New shapes can be added without changing the existing code.
- \* The correct ~~shape~~ area() is selected at runtime.
- \* This improves flexibility and extensibility.
- \* The small disadvantage is vtable/vptr overhead, which adds a little memory usage and function-call overhead.

## 10) Exception Handling in File processing

### i) use of Try catch

- \* try contains file processing code.
- \* catch handles errors like file not found, invalid data and empty files.
- \* Program can continue processing other files.

### ii) Custom Exception Classes.

- Custom exceptions give clear error messages such as :
  - "student file is empty"
  - "Invalid student record format"
  - "File not found"

### iii) Why Exception Handling is Better than Return code ?

- \* Keeps error handling separate.
- \* Makes code clean and easy to maintain.
- \* prevents program crashes
- \* Handles errors effectively.